

Final Exam Topics

13. Theory of computation

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Theory of computation

The Turing machine and the Church-Turing thesis. Variants of Turing machines: multitape, nondeterministic, counting, offline. Recursive and recursively enumerable languages. Undecidable problems. Time and space complexity classes: P, NP, PSPACE. NP-complete problems.

1 Computability

1.0.1 Algorithm models

- **Gödel:** recursive functions (primitive recursive functions in 1931, then more general in 1934)
- **Church:** λ -calculus, λ -definable functions: equivalent to the recursive functions (proven)
- **Turing:** Turing machine (1936); the λ -definable and the Turing-machine-computable functions coincide (proven)

Church-Turing thesis: The various mathematical models of computability all define the class of effectively computable functions.

1.0.2 Concepts

By a computational problem we mean a question formulated in the language of mathematics to which we want to give the answer with an algorithm. To almost every problem of practical life, using an appropriate abstraction, a computational problem can be assigned.

A problem together with its associated concrete input is called an instance of the problem.

A special computational problem is the decision problem. In this case the question related to the problem is a yes/no question, so for an instance of the problem the answer will be “yes” or “no”.

A computational problem can be represented by a function $f : A \rightarrow B$. The set A contains the individual inputs of the problem, typically encoded in a word over an appropriate alphabet, while the set B contains the answers given to the inputs, likewise encoded in a word over some suitable alphabet. Naturally, if it is a decision problem, then the range of f , that is, B , is a two-element set: $\{yes, no\}$, $\{1, 0\}$, etc.

Computable function: A function $f : A \rightarrow B$ is called *computable* if for every element $x \in A$ the function value $f(x) \in B$ is computable by some algorithmic model.

Solvable, decidable problem: A computational problem is *solvable* (in the case of a decision problem we say that it is *decidable*) if the function determined by it is computable.

Time requirement of algorithms: Let $f, g : \mathbb{N} \rightarrow \mathbb{N}$ be functions, where $\mathbb{N}$ is the set of natural numbers. We say that f grows at most as fast as g (notation: $f(n) = \mathcal{O}(g(n))$) if $\exists c > 0$ and $n_0 \in \mathbb{N}$ such that $f(n) \leq c * g(n) \forall n \geq n_0$. The notation $f(n) = \Omega(g(n))$ denotes that $g(n) = \mathcal{O}(f(n))$ holds, and $f(n) = \Theta(g(n))$ denotes that both $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$ hold.

Example: $3n^3 + 5n^2 + 6 = \mathcal{O}(n^3)$, $n^k = \mathcal{O}(2^n) \forall k \geq 0$, etc.

Theorem: Every polynomial function grows more slowly than any exponential function, that is, for every polynomial $p(n)$ and every $c > 0$ there is an integer n_0 such that for $\forall n \geq n_0$, $p(n) \leq 2^{cn}$

Correspondence of a computational problem to a decision problem: Consider a computational problem P and let $f : A \rightarrow B$ be the function determined by P . Then a decision problem P' can be given for P such that P' is decidable exactly when P is computable. Namely, let us pair, for every element $a \in A$, the elements a and $f(a)$, and encode the pairs thus obtained each in a word. After this let P' be the formal language formed from the words thus obtained. It is obvious that if for every element $a \in A$ and $b \in B$ the containment $(a, b) \in P'$ is decidable (that is, P' is decidable), then P is computable and vice versa. Because of this correspondence, in what follows we typically deal with decision problems.

2 Turing machines

Similarly to the finite automaton or the pushdown automaton, the Turing machine is also a device with finitely many states. The Turing machine works on a two-way infinite tape. The tape is divided into cells; this is essentially the (unlimited) memory of the machine. Initially on the tape there is only the input word, one letter on each cell. The other cells of the tape are filled with so-called blank or space ($\sqcup$) symbols. Initially the so-called read-write head of the machine stands on the first letter of the input word and the machine is in its initial state. The machine can move the read-write head arbitrarily on the tape. It is furthermore able to read out and rewrite the content of the tape at the head position. The machine has two distinguished states, the states q_i and q_n . If it gets into these states, then it accepts respectively rejects the input word. Formally we define the Turing machine in the following way.

The formal definition of the Turing machine: The Turing machine is a system $M = (Q, \Sigma, \Gamma, \delta, q_0, q_i, q_n)$, where:

- Q is the finite, nonempty set of states,
- $q_0, q_i, q_n \in Q$, q_0 is the initial state, q_i the accepting state, and q_n the rejecting state,
- Σ and Γ are alphabets, the alphabet of the input symbols and of the tape symbols such that $\Sigma \subseteq \Gamma$ and $\Gamma - \Sigma$ contains a special $\sqcup$ symbol,
- $\delta : (Q - \{q_i, q_n\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ is the transition function.

As in the case of pushdown automata, the phases of operation of a Turing machine M can also be described with configurations.

Configuration transition of a Turing machine: The configuration transition of M is a relation $\vdash \subseteq \mathcal{C}_M \times \mathcal{C}_M$, which we define as follows. Let $uqav$ be a configuration where $a \in \Gamma$ and $u, v \in \Gamma^*$. We distinguish the following three cases:

- If $q \in \{q_i, q_n\}$, then we say that the configuration uqv is a halting configuration. Furthermore, in the case $q = q_i$ we speak of an accepting, and in the case $q = q_n$ of a rejecting configuration.

Turing-recognizable language, the class of recursively enumerable languages: A language $L \subseteq \Sigma^*$ is Turing-recognizable if $L = L(M)$ for some Turing machine M . Turing-recognizable languages are also usually

called *recursively enumerable*. The class of recursively enumerable languages is denoted by RE .

Turing-decidable language, the class of recursive languages: A language $L \subseteq \Sigma^*$ is Turing-decidable if there exists a Turing machine that on every input reaches a halting configuration and recognizes L . Turing-decidable languages are also usually called *recursive*. The class of recursive languages is denoted by R .

Running time, time requirement of a Turing machine: Consider a Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, q_i, q_n)$ and one of its input words $u \in \Sigma^*$. We say that the running time (time requirement) of M on the word u is n ($n \geq 0$), if M can reach a halting configuration in n steps from the initial configuration $q_0 u \sqcup$. If there is no such number, then the running time of M on the word u is infinite.

Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. We say that the time requirement of M is $f(n)$ (or that M is an $f(n)$ time-bounded machine) if for every input word $u \in \Sigma^*$ the time requirement of M on the word u is at most $f(l(u))$.

2.0.1 Multitape Turing machines

Multitape Turing machines, naturally, have more than one tape. To each tape belongs a read-write head, which can move on the tape independently of each other.

Definition of a multitape Turing machine: Let $k > 1$. A k -tape Turing machine is a system $M = (Q, \Sigma, \Gamma, \delta, q_0, q_i, q_n)$, where the components except for δ coincide with the components of the single-tape Turing machine, and δ is given as follows. $\delta : (Q - \{q_i, q_n\}) \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$. Let $q, p \in Q$, $a_1, a_2, \dots, a_k, b_1, b_2, \dots, b_k \in \Gamma$ and $D_1, D_2, \dots, D_k \in \{L, R, S\}$. If $\delta(q, a_1, a_2, \dots, a_k) = (p, b_1, b_2, \dots, b_k, D_1, D_2, \dots, D_k)$, then the machine, from state q , if on its tapes it reads respectively the letters $a_1, a_2, \dots, a_k$, can transition into state p , while it rewrites the letters $a_1, a_2, \dots, a_k$ to the letters $b_1, b_2, \dots, b_k$ and moves the heads on the tapes in the $D_1, D_2, \dots, D_k$ directions.

The definition of the configuration of the multitape Turing machine, the configuration transition, as well as the recognized or decided language is the natural generalization of the single-tape case. We also define the time requirement of the multitape Turing machine similarly to the single-tape one.

Equivalence of multitape and single-tape machines: For every k -tape, $f(n)$ time-bounded Turing machine there is an equivalent $\mathcal{O}(n * f(n))$ time-bounded single-tape Turing machine.

2.0.2 Nondeterministic Turing machines

The state function of a nondeterministic Turing machine M is of the form $\delta : (Q - \{q_i, q_n\}) \times \mathcal{P}(\Gamma \rightarrow Q \times \Gamma \times \{L, R\})$. So M can transition from each of its configurations into some (possibly zero) different configurations. In this way the computation sequences of M on a word u can be represented by a tree. The root of the tree is the initial configuration of M , and its vertices are the configurations of M . Every leaf of the tree corresponds to a computation sequence of M on u . M accepts u if some leaf of the tree is an accepting configuration. Let us call the tree just described the nondeterministic computation tree of M on u . The language recognized by M can be defined similarly to the deterministic case, and the language decided by the machine as follows.

Language decided by a nondeterministic Turing machine: We say that a nondeterministic Turing machine M decides a language $L \subseteq \Sigma^*$ if it recognizes it, and for every word $u \in \Sigma^*$ the computation sequences of M are finite and lead to an accepting or rejecting configuration.

Time requirement of a nondeterministic Turing machine: Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function, M a nondeterministic Turing machine. The time requirement of M is $f(n)$ if on an input u of length n there are no computation sequences of M longer than $f(n)$, that is, the computation tree of M on u is at most $f(n)$ high.

Equivalence of deterministic and nondeterministic Turing machines: For every nondeterministic Turing machine M an equivalent deterministic Turing machine M' can be given. Furthermore, if M is of time requirement $f(n)$ for some function $f : \mathbb{N} \rightarrow \mathbb{N}$, then M' is of time requirement $2^{\mathcal{O}(f(n))}$.

3 Undecidable problems

In this chapter we show that although the Turing machine is the most general possible algorithm model, there are nevertheless problems that cannot be computed with a Turing machine.

Reminder: The class of recursively enumerable (Turing-recognizable) languages is denoted by RE , the class of recursive (Turing-decidable) languages by R .

It is clear that $R \subseteq RE$. Our goal is to show that R is a proper subset of RE , that is, there is a language (problem) that is Turing-recognizable but not decidable.

We will examine only those Turing machines whose input alphabet is the set $\{0, 1\}$. This does not mean a restriction of generality, since if we find a language over $\{0, 1\}$ that cannot be decided with such a Turing machine, then this language cannot be decided at all.

3.0.1 Encoding of Turing machines

The words over $\{0, 1\}$ can be enumerated (that is, they are countable). Indeed, consider the enumeration in which the words follow each other by their length, and of two words of equal length the one that precedes the other in alphabetical order comes first. In this way an enumeration of the elements of the set $\{0, 1\}^*$ takes the following shape: $w_1 = \varepsilon$, $w_2 = 0$, $w_3 = 1$, $w_4 = 00$, $w_5 = 01$ and so on. In this chapter, therefore, by the word w_i we denote the i -th element of $\{0, 1\}^*$.

Let furthermore M be a Turing machine over the input alphabet $\{0, 1\}$. There is a number $k > 0$ such that Q can be written in the form $Q = \{p_1, \dots, p_k\}$, where $p_1 = q_0$, $p_{k-1} = q_i$, $p_k = q_n$. Furthermore, there is a number $m > 0$ such that Γ can be written in the form $\Gamma = \{X_1, \dots, X_m\}$, where $X_1 = 0$, $X_2 = 1$, $X_3 = \sqcup$, and $X_4, \dots, X_m$ are the further tape symbols of M . Let us finally call the symbols L, R, S (which denote directions) respectively D_1, D_2 and D_3 . After this a transition $\delta(p_i, X_j) = (p_r, X_s, D_t)$ ($0 \leq i, r \leq k$, $1 \leq j, s \leq m$ and $1 \leq t \leq 3$) of M can be encoded with the word $0^i 10^j 10^r 10^s 10^t$. Since the length of every 0-block is at least 1, in the word encoding the transition the subword 11 does not appear. So the words encoding all transitions of M can be concatenated into a word in which the transitions are separated from each other by the subword 11. The word thus obtained encodes M itself.

In what follows we denote by M_i the Turing machine encoded by the word w_i ($i \geq 1$). If w_i is not the encoding of a Turing machine described above, then let us regard M_i as one that, for every input, immediately goes into the state q_n , that is, $L(M_i) = \emptyset$.

Later on we will need to encode a pair of a Turing machine and input (M, w) in a word over $\{0, 1\}$. Since the encoding of Turing machines cannot contain 111, therefore the encoding of (M, w) is the following: after the encoding of M we write 111, then after it w .

3.0.2 A non-recursively-enumerable language

The L_{diag} language: The L_{diag} language contains the binary encodings of those Turing machines over $\{0, 1\}$ that do not accept their own encoding as the input word, that is, $L_{diag} = \{w_i \mid i \geq 1, w_i \notin L(M_i)\}$

Theorem: $L_{diag} \notin RE$.

3.0.3 A recursively enumerable but undecidable language

The L_u language: Consider the set of those pairs (M, w) (encoded in an appropriate binary word), where M is a Turing machine over the input alphabet $\{0, 1\}$, and w is a word over $\{0, 1\}$ such that $w \in L(M)$, that is, M accepts w . We denote this language by L_u . $L_u = \{\langle w_i, w_j \rangle \mid i, j \geq 1, w_j \in L(M_i)\}$

Theorem: $L_u \in RE$.

Theorem: $L_u \notin R$.

3.0.4 Further theorems

1. Let L be a language. If $L, \bar{L} \in RE$, then $L \in R$. Consequence: the recursively enumerable languages are not closed under complementation.
2. If $L \in R$, then $\bar{L} \in R$, that is, the recursive languages are closed under complementation.

3.0.5 Further undecidable problems

Computable function: Let Σ and Δ be two alphabets and f a function mapping from Σ^* into Δ^* . We say that f is computable if there is a Turing machine M such that, starting M with a word $w \in \Sigma^*$ on its input, M halts so that on its tape there is the word $f(w) \in \Delta^*$.

Reduction of decision problems: Let $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Delta^*$ be two decision problems. L_1 is reducible to L_2 ($L_1 \leq L_2$) if there is a computable function $f : \Sigma^* \rightarrow \Delta^*$ such that for every word $w \in \Sigma^*$, $w \in L_1$ holds exactly when $f(w) \in L_2$ also holds.

Theorem: Let $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Delta^*$ be two decision problems and suppose that L_1 is reducible to L_2 . Then the following statements are true:

1. If L_1 is undecidable, then so is L_2 .
2. If $L_1 \notin RE$, then $L_2 \notin RE$.

The halting problem: Let $L_h = \{\langle M, w \rangle \mid M \text{ halts on the input } w\}$, that is, L_h contains, encoded, those Turing-machine-and-input pairs $\langle M, w \rangle$ for which M halts on the input w . L_h is undecidable (L_u is reducible to L_h), but $L_h \in RE$.

The L_{empty} problem: Let $L_{empty} = \{\langle M \rangle \mid L(M) = \emptyset\}$. L_{empty} is undecidable (L_u is reducible to L_{empty}), and $L_{empty} \notin RE$.

(Non-trivial) property of recursively enumerable languages: If $\mathcal{P}$ is a set of recursively enumerable languages, then $\mathcal{P}$ is a property of the recursively enumerable languages. If $\mathcal{P} \neq \emptyset$ and $\mathcal{P} \neq RE$, then $\mathcal{P}$ is a non-trivial property of the recursively enumerable languages.

Rice's theorem: For a given property $\mathcal{P}$ let us denote by $L_{\mathcal{P}}$ the set of encodings of those Turing machines that recognize a language in $\mathcal{P}$. If $\mathcal{P}$ is a non-trivial property of the recursively enumerable languages, then $L_{\mathcal{P}}$ is undecidable.

Post Correspondence Problem (briefly PCP): We define the PCP problem as follows. Let Σ be an alphabet containing at least two letters and let $D = \left\{ \left[\frac{u_1}{v_1} \right], \dots, \left[\frac{u_n}{v_n} \right] \right\}$ be a set of dominoes, in which $n \geq 1$ and $u_1, \dots, u_n, v_1, \dots, v_n \in \Sigma^+$. The question is whether there is an index sequence $1 \leq i_1, \dots, i_m \leq n$ ($m \geq 1$) for which

it holds that, writing the dominoes $[\frac{u_{i_1}}{v_{i_1}}], \dots, [\frac{u_{i_m}}{v_{i_m}}]$ next to each other, the same word is obtained below and above, that is, $u_{i_1} \dots u_{i_m} = v_{i_1} \dots v_{i_m}$. In this case the above domino sequence is called a solution of D .

As a formal language we can define the PCP as follows: $\text{PCP} = \{\langle D \rangle \mid D \text{ has a solution}\}$. PCP is undecidable.

4 Complexity theory

The goal of complexity theory is the classification of solvable (and within these, decidable) problems according to the amount of resources (typically time and space) needed for the solution.

4.0.1 Time complexity concepts

TIME: Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. $\text{TIME}(f(n)) = \{L \mid L \text{ is decidable by an } \mathcal{O}(f(n)) \text{ time Turing machine}\}$

$\mathbf{P} = \bigcup_{k \geq 1} \text{TIME}(n^k)$. So $\mathbf{P}$ contains those languages that are decidable by a polynomial-time-bounded deterministic Turing machine. Such is, for example, the well-known REACHABILITY problem, whose input is a graph G and its two distinguished vertices (s and t). The question is whether there is a path in G from s to t . If we regard the REACHABILITY problem as a language, then we can write that

$$\text{REACHABILITY} = \{\langle G, s, t \rangle \mid \text{there is a path in } G \text{ from } s \text{ to } t\}.$$

A deterministic Turing machine deciding the REACHABILITY problem in polynomial time can easily be given, so $\text{REACHABILITY} \in \mathbf{P}$.

NTIME: Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function.

$\text{NTIME}(f(n)) = \{L \mid L \text{ is decidable by an } \mathcal{O}(f(n)) \text{ time nondeterministic Turing machine}\}$

$\mathbf{NP} = \bigcup_{k \geq 1} \text{NTIME}(n^k)$. The problems in $\mathbf{NP}$ have a common property in the following sense. If we consider an instance of a problem in $\mathbf{NP}$ and a possible “proof” that this instance is a “yes” instance of the given problem, then the checking of the correctness of this proof can be done in polynomial time. Accordingly, a nondeterministic Turing machine deciding a problem in $\mathbf{NP}$ generally works so that it “guesses” a possible solution of the problem’s input, and checks in polynomial time whether the solution is correct.

Consider the SAT problem, which we define as follows. Given a propositional-logic CNF ϕ . The question is whether it is satisfiable. The proof that ϕ is satisfiable is a variable assignment under which, evaluating ϕ , we get a true value. An arbitrary variable assignment is therefore a possible proof of the satisfiability of ϕ . And the checking of whether this assignment really makes ϕ true can be done in polynomial time. SAT is a problem in $\mathbf{NP}$.

It follows from the definitions that the containment $\mathbf{P} \subseteq \mathbf{NP}$ holds.

4.0.2 NP-complete problems

Polynomial-time computable function: Let Σ and Δ be two alphabets and f a function mapping from Σ^* into Δ^* . We say that f is polynomial-time computable if it is computable by a polynomial-time Turing machine.

Polynomial-time reduction of decision problems: Let $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Delta^*$ be two decision problems. L_1 is polynomial-time reducible to L_2 ($L_1 \leq_p L_2$) if $L_1 \leq L_2$ and the function f used in the reduction is polynomial-time computable.

Theorem: Let L_1 and L_2 be two problems such that $L_1 \leq_p L_2$. If L_2 is

1. in $\mathbf{P}$, then L_1 is also in $\mathbf{P}$.
2. in $\mathbf{NP}$, then L_1 is also in $\mathbf{NP}$.

NP-complete problem: Let L be a problem. We say that L is **NP**-complete if

1. it is in **NP**, and
2. every further problem in **NP** is polynomial-time reducible to L .

Theorem: Let L be an **NP**-complete problem. If $L \in \mathbf{P}$, then $\mathbf{P} = \mathbf{NP}$.

Remark: Currently we do **NOT** know of any **NP**-complete problem in **P**!!!

Theorem: Let L_1 be an **NP**-complete, and L_2 a problem in **NP**. If $L_1 \leq_p L_2$, then L_2 is also **NP**-complete.

Cook's theorem: SAT is **NP**-complete.

Let $k \geq 1$. $k\text{SAT} = \{\langle \phi \rangle \mid \text{in every clause of } \phi \text{ there are } k \text{ literals.}\}$

Theorem: 3SAT is **NP**-complete, since $\text{SAT} \leq_p 3\text{SAT}$.

$\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ is a finite graph, } k \geq 1, G \text{ has a subgraph with } k \text{ vertices}\}$. So **CLIQUE** contains those pairs G and k , encoded in words over an appropriate alphabet, for which it holds that in G there is a complete subgraph with k vertices, that is, a subgraph in which there is an edge between any two vertices.

$\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ is a finite graph, } k \geq 1, G \text{ has a subgraph with } k \text{ vertices}\}$. So **CLIQUE** contains those pairs G and k , encoded in words over an appropriate alphabet, for which it holds that in G there is a complete subgraph with k vertices, that is, a subgraph in which there is an edge between any two vertices.

$\text{INDEPENDENT SET} = \{\langle G, k \rangle \mid G \text{ is a finite graph, } k \geq 1, G \text{ has an independent vertex set of } k \text{ elements}\}$. That is, **INDEPENDENT SET** contains those pairs G and k for which it holds that in G there are k vertices, none of which is connected to another.

$\text{VERTEX COVER} = \left\{ \langle G, k \rangle \mid \begin{array}{l} G \text{ is a finite graph, } k \geq 1, G \text{ has a vertex set of } k \text{ elements,} \\ \text{which contains at least 1 endpoint of every edge of } G. \end{array} \right\}$.

CLIQUE, **INDEPENDENT SET** and **VERTEX COVER** are **NP**-complete ($\text{CLIQUE} \leq_p \text{INDEPENDENT SET} \leq_p \text{VERTEX COVER}$).

$\text{TRAVELLING SALESMAN} = \left\{ \langle G, k \rangle \mid \begin{array}{l} G \text{ is a finite undirected graph, on the edges a positive integer weight each, and} \\ \text{there is in } G \text{ a Hamiltonian cycle of total weight at most } k \end{array} \right\}$.

Theorem: The **TRAVELLING SALESMAN** problem is **NP**-complete.

4.0.3 Space complexity

We examine space complexity on a special, so-called offline Turing machine.

Off-line Turing machine: By an offline Turing machine we mean a multitape Turing machine that can only read the tape containing the input, and can also write on the other, so-called work tapes. Into the space requirement of the offline Turing machine only the area used on the work tapes is counted.

In what follows, by Turing machine we always mean an offline Turing machine. Now let us define the language classes related to space complexity.

SPACE($f(n)$) = $\{L \mid L \text{ is decidable by an } \mathcal{O}(f(n)) \text{ space deterministic Turing machine}\}$

NSPACE($f(n)$) = $\{L \mid L \text{ is decidable by an } \mathcal{O}(f(n)) \text{ space nondeterministic Turing machine}\}$

PSPACE = $\bigcup_{k>0} \mathbf{SPACE}(n^k)$

NPSPACE = $\bigcup_{k>0} \mathbf{NSPACE}(n^k)$

L = **SPACE**($\log_2 n$)

NL = **NSPACE**($\log_2 n$)

Savitch's theorem: If $f(n) \geq \log n$, then $\mathbf{NSPACE}(f(n)) \subseteq \mathbf{SPACE}(f^2(n))$.